

UNIT I: Introduction & Lexical Analysis

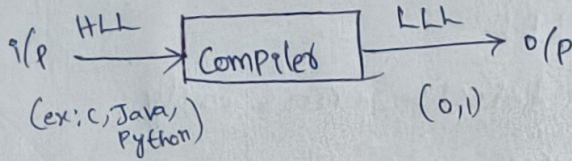
This unit is purely theoretical and easy to score. Do not miss the diagrams.

Long Answer (10 Marks):

1. **The Holy Grail Question:** Explain the Phases of a Compiler in detail with a neat diagram. Trace the statement `position = initial ÷ rate * 60` through all phases. (*This question appears in almost every other paper*).
2. Explain the Input Buffering techniques (Sentinel, Pair of Buffers) in Lexical Analysis.
3. Discuss the structure of the LEX tool and its specifications.
4. Construct a DFA from a Regular Expression (Practice Thompson's Construction and Subset Construction method).

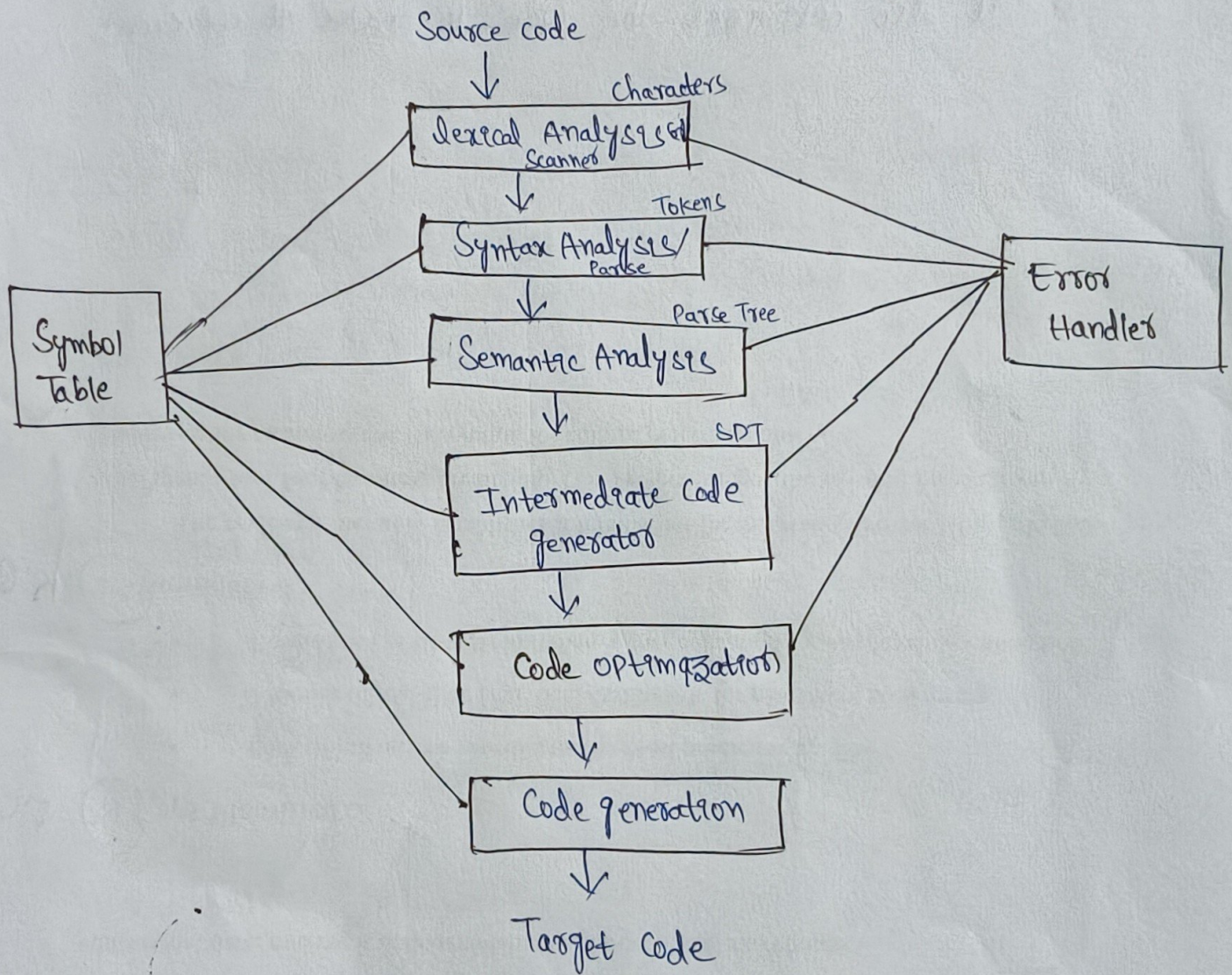
UNIT-4

HLL - High Level Language
LLH - Low Level Language



→ Compiler translates HLL to LLH. (or) converts Source code to machine level language

* Phases of Compiler



- Symbol Table interacts with all the phases,
- Error Handler interacts with all the phases,
- Each phase receives i/p from its previous phase
- and produces o/p to the next phase

① 2
1. Lexical Analysis :- Takes input as characters from Source code.

- Gives output in the form of Tokens / Lexemes

- It receives input from the source codes, and produce output to the Syntax Analyzer.

- Also called as Scanner (or) Tokenizer

- It reads the Source Program character by character & produces a stream of tokens / lexemes

- A token may be identifier, operator, literal, constant

- breaks input code into tokens

- removes comments, space.

ex: `int a = 5;` $\Rightarrow$ `int, a, =, 5, ;`

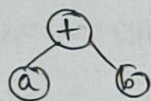
2. Syntax Analyzer: It checks the syntax of tokens.

- If errors found, error handler report to the user.

- If it ~~is~~ doesn't contains errors, then produces Parse tree as o/p.

- Also called as (Parser)

ex: `a + b`



- For Parse tree, operator is ~~also~~ root node

- children are Identifiers

- takes input as tokens, produces o/p as Parse tree / Syntax tree.

- Checks whether tokens follow the grammar of the language [`a = ;` Syntax error]

3. Semantic Analysis: It checks meaning of Parse tree

- Checks meaning of the program.

- rules like: Type checking, Variable declaration.

ex: `int x;`

`x = 'hello';` $\rightarrow$ Semantic error

4. ICG: It generates intermediate code.

- It uses 3-address code [max 3 operators and 1]

5. Code optimization: Complex $\rightarrow$ Simple

$\rightarrow$ ~~Process~~ Process of improving the intermediate code

- making the code better, faster & lightweight, removes duplicates.
- Removing unnecessary work from the program.

ex: $x = 2 * 8 \Rightarrow x = 16$

6. Code Generation:

$\rightarrow$ It generates machine code.

$\rightarrow$ Converts optimized intermediate code to machine code

ex: $a = b + c * 60$

~~Phase~~ Phase: read one by one, convert to Token

Lexeme	Token
a	identifier
=	assignment operator
b	identifier
+	addition operator
c	identifier
*	multiplication operator
60	constant

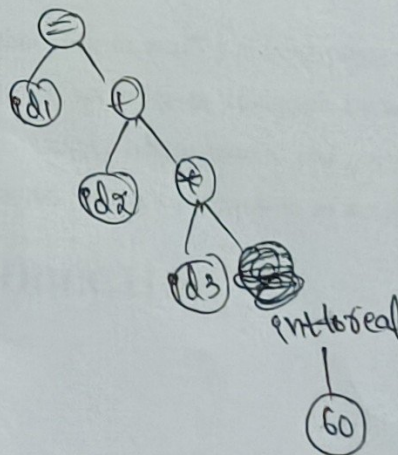
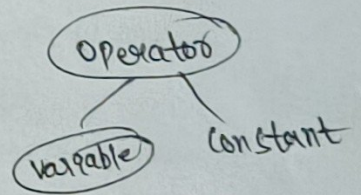
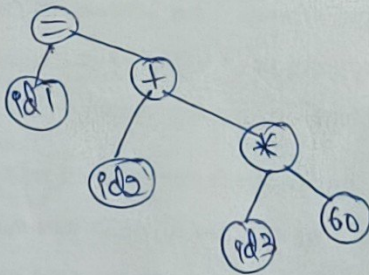
Token has: (Token name, Token value)
 $\Rightarrow \langle id, 1 \rangle, \langle = \rangle, \langle id, 2 \rangle, \langle + \rangle,$
 $\langle id, 3 \rangle, \langle * \rangle, \langle 60 \rangle$

For operators, constant, ~~do~~ $\rightarrow$ no Token value

O/p: $id1 = id2 + id3 * 60$

~~Phase~~ Phase: checks for Syntax errors then construct Parse tree

r/p: $id1 = id2 + id3 * 60$



3rd phase:

$$t_1 = \text{initial (GO)}$$

$$t_2 = id_3 * t_1$$

$$t_3 = id_2 + t_2$$

$$id_1 = t_3$$

Lexical Analyzer Generator

- LEX is a tool
- LEX is Lexical Analyzer Generator
- It is used to automatically create a 'lexical analyzer' / Scanner.
- Fast creation of lexical analyzers.
- Reduces programmer effort
- A LEX has 3 sections.
- each section is separated by %%

Definition Section

%%

Rule Section

%%

User Code Section

- Fast & automatic
- easy
- Less manual code
- Uses

Applications

- Tokenizing
- removing comments, white space
- Data filtering
- Preprocessing

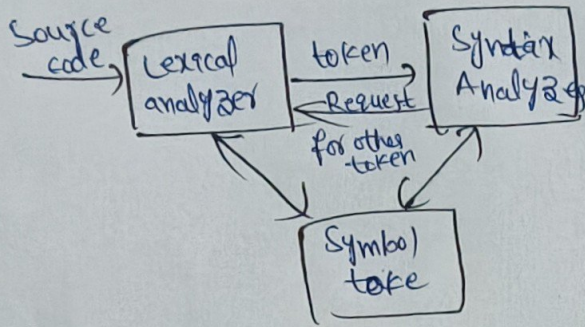
The Role of a Lexical Analyser

→ Also called Scanner

→ First phase of compiler.

→ reads program character by character.

→ convert to tokens



Token: It is a meaningful unit in a program

ex: id, number, [keyword (keywords)], +, -, *, (,), ' ,

→ each token has:

Token name → eg: id, number, operator, plus

Token value (lexeme) → eg: x, 12, +

1. Tokenization: Breaks the input source code into tokens

ex: `int a=5;` becomes $(\langle \text{int} \rangle \langle x \rangle \langle = \rangle \langle 5 \rangle \langle ; \rangle)$
`int, x, =, 5, ;`

2. Removes whitespaces & comments:

- eliminates: spaces, tabs, newlines, comments

3. Handles Errors: detects errors, & solve it.

4. Communicates with Parser

• Parser = Syntax analyzer

• Parser asks: Give me next token

• Lexical analyzer gives token

5. Symbol Table management; • collects identifiers (id) & inserts to

Symbol Table

`int a=5;`

id	a

Input Buffering:

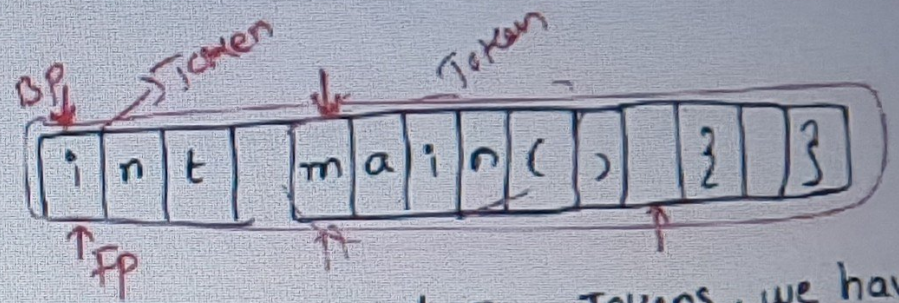
* The Lexical Analyzer scans the input from left to right one character at a time and produces a stream of Tokens

→ In order to read the Tokens, the Lexical Analyzer mainly uses 2 pointers

- ① Lexeme Begin pointer ② Forward pointer

A pointer points to a variable, stores the address of a variable.

- During compilation, the program resides in hard disk
- Whenever a program is saved, then the program is stored in the hard disk, during compilation also the program resides in the hard disk, at the time of execution, the program is placed in the main memory, because a program is executed by CPU
- CPU can execute a program only when the program is placed in main memory



In order to read the Tokens, we have two pointers

① Lexeme Begin pointer: It points to the beginning character of the current lexeme.

② Forward pointer: It is placed at the beginning point of the lexeme. It reads the first character & moves ahead (right) in order to read the next character

⇒ Whenever forward pointer reads the blank space then it identifies that it is the end of the lexeme

⇒ Forward pointer ignores the blank space & places the Lexeme Begin pointer and Forward pointer is placed at next location

→ The problem is corresponding program is stored in the hard disk, we can read only one character at a time from the hard disk. If the program size is very large, then it requires too many system calls, in order to read the data from the corresponding hard disk.

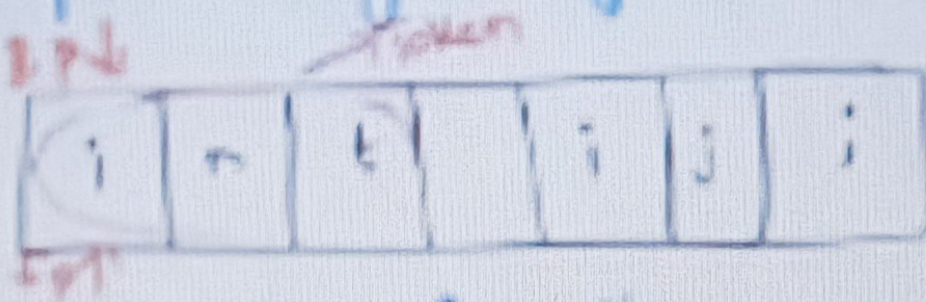
→ We can overcome this problem with the help of Buffering technique

Buffering Techniques:

The use of Buffering is a block of characters are to be read into the buffer using only one system call.

① one Buffer scheme:

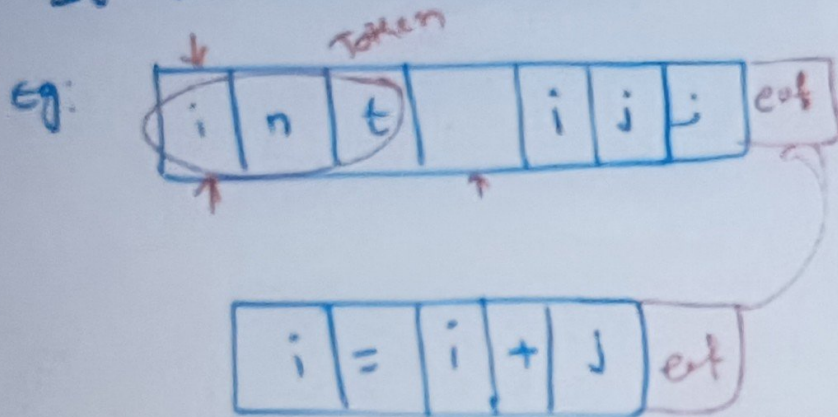
→ It uses only one buffer to read the input string. eg int i, j, k



→ The problem is if our input string size is larger than the capacity of the buffer then the buffer has to be overwritten in order to read the remaining input string.

(ii) Two-Buffer Scheme:

It maintain two buffers



⇒ In This method two Buffers are used to store The Input string, The first buffer and second Buffers are Scanned alternatively.

⇒ The Beginning pointer & Forward pointer are pointed to The first character of The first buffer.

⇒ After The first Buffer is Completely filled Then we have to Start filling of Second Buffer.

→ Here eof is a special character which is not a source program

→ whenever two pointers encounter 'eof' It recognizes First Buffer is Completely filled So now filling of second Buffer will start.

→ whenever The Second Buffer is Completely filled Then The remaining Input string has to be filled in The first Buffer.
Then First Buffer data should be overridden.